

COMP90015 · DISTRIBUTED SYSTEMS

A Multi-Threaded Distributed Subject Enrolment System

ASSIGNMENT 1 — DESIGN REPORT

Nikhil Singh

Student ID: 1461600

School of Computing and Information Systems

The University of Melbourne

Java 8+ · TCP sockets and threads only · no RMI, no external libraries

1 Problem context

A university's enrolment records are a single shared resource that many administrators act on at the same time from different machines. Each subject has a fixed capacity and a list of enrolled students, and every interesting operation — enrol, withdraw, transfer, change capacity — is a *read-modify-write* sequence over that state. Run naively, two administrators claiming the last remaining seat both read "one seat free", both write, and the subject ends up oversubscribed; a transfer implemented as a withdraw followed by an enrol leaves the student visible in both subjects, or in neither, for an observable interval.

This system is a client-server solution to that problem. A single server process owns the authoritative data; remote clients hold none of it and change it only by request. The server must serve many clients at once without serialising unrelated work, while making each operation atomic with respect to the subject(s) it touches: reads of one subject proceed in parallel, writes to one subject are exclusive, and work on different subjects does not interact at all. A configurable artificial delay, held across the locks, makes those properties externally observable rather than merely asserted, and every acknowledged write must survive a restart of the server.

2 Component description

The system is two runnable jars built with plain `javac`: `SubjectServer.jar` and `SubjectClient.jar`. They speak one JSON object per line over TCP, terminated by `\n`, with exactly one response line per request line on the same connection. Message framing is the newline: both sides read with `BufferedReader.readLine()`, so however TCP splits or coalesces segments, one line is reassembled into exactly one message.

Server — `SubjectServer`

- **Startup and loading.** Validates port, data file and delay, then loads subjects: a state file left by a previous run wins, otherwise the supplied data file is read. Loading enforces the data-file rules (unique `subjectCode`, positive `capacity`, `enrolledStudentIds.length ≤ capacity`) and exits naming the offending entry rather than starting from invalid data.
- **Accept loop.** Binds a `ServerSocket` and, for each accepted connection, registers a `ConnectionHandler` and starts a dedicated thread named `client-N`.
- **Connection handler** (inner class; one instance and one thread per client). Loops on `readLine()`, passes each line to `process()`, writes back the response it returns. A malformed request is answered and the connection kept open; only an I/O failure, the client hanging up, or shutdown ends the loop.
- **Dispatch and business logic.** `process()` parses the line and dispatches on `op` to `query`, `enrol`, `withdraw`, `transfer` or `updateCapacity`. Each acquires the appropriate lock(s), sleeps for the artificial delay, applies the specified status ordering (`NOT_FOUND`, `INVALID_REQUEST`, `NOT_ENROLLED`, `DUPLICATE_ENROLMENT`, `FULL`), mutates state and releases the lock(s). `process()` never throws: a malformed request becomes `INVALID_REQUEST` and an unexpected runtime exception becomes `ERROR`, so one bad request can kill neither a connection nor the server.
- **Persistence.** Every mutation records its subject's committed snapshot while still under the write lock; the state file is then rewritten from those snapshots *before* the response is sent, via a temporary file moved into place with `ATOMIC_MOVE`.
- **Console and logging.** A separate non-daemon console thread reads `status` (active connections, in-flight operations) and `quit` (graceful shutdown). Each completed operation logs one line: timestamp, thread name, operation, subject, status.

Client — `SubjectClient`

- Connects once at startup, reporting and exiting if the server is unreachable, then runs an interactive command loop: `query`, `enrol`, `withdraw`, `transfer`, `update`, `quit`. Bad input (wrong argument count, non-integer capacity) is rejected locally without contacting the server.
- A recognised command becomes one request built by `ProtocolMessage`; `exchange()` writes it, flushes, and blocks on a single `readLine()` for the reply.
- Replies are summarised for the user: `QUERY` prints `enrolledCount/capacity` and the student list, any non-success reply prints as `STATUS: message`. If the server hangs up or the reply will not parse, the client says so and returns to its prompt instead of dying.

Shared and supporting classes

- `Subject` — one subject's `subjectCode`, `capacity` and `LinkedHashSet` of enrolled IDs, *plus the `ReentrantReadWriteLock` that guards them*.
- `ProtocolMessage`, `ProtocolException`, `SimpleJson` (provided) — building, parsing and validating the wire format; `SimpleJson` is reused to encode the state file.

3 Class design and interaction

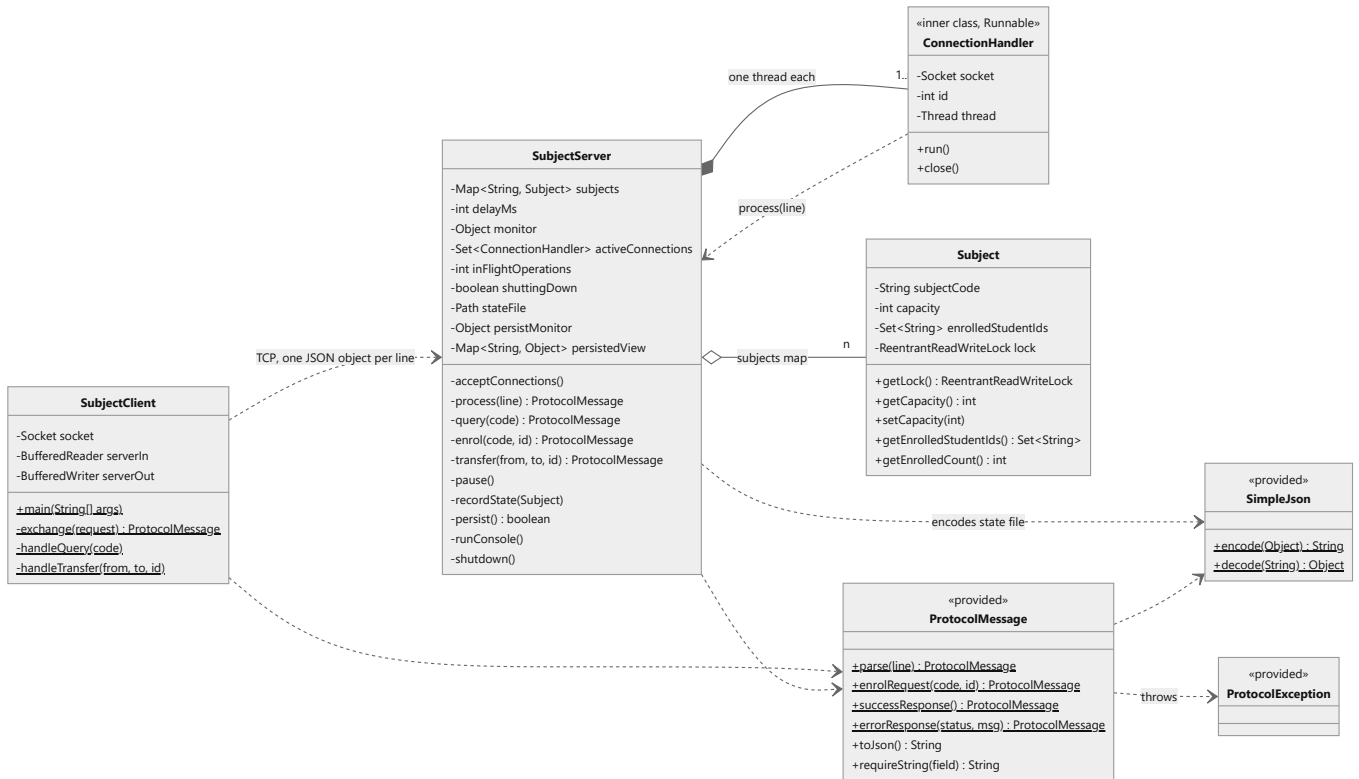


Figure 1 — Class design. Each **Subject** carries the lock that guards its own state, so the server needs no separate lock table.

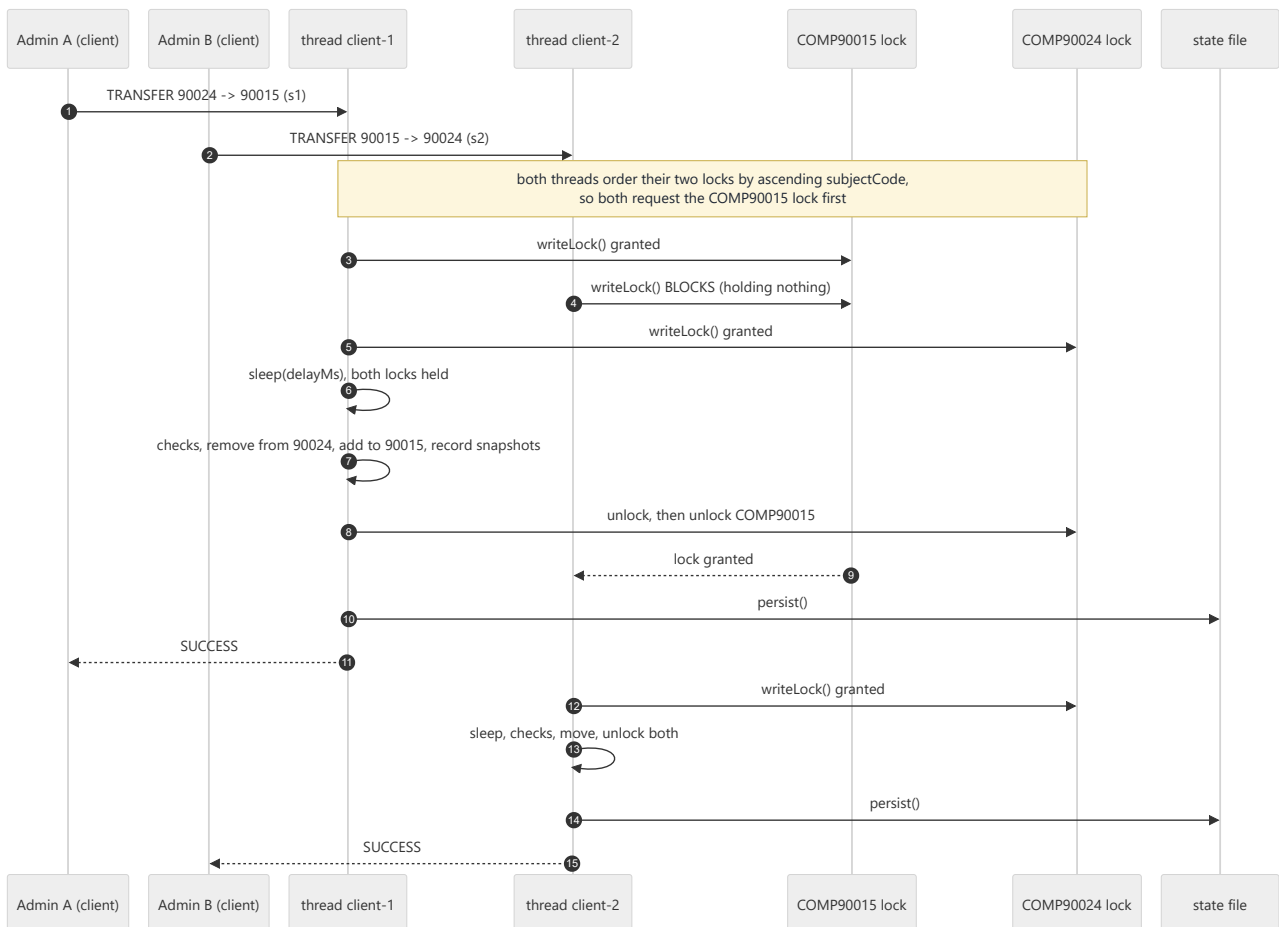


Figure 2 — Two opposing **TRANSFERS** on the same pair of subjects. The global lock order turns a potential circular wait into a plain queue; the delay elapses inside the critical section, and the state file is written after the locks are released but before the client is told **SUCCESS**.

4 Critical analysis

4.1 Design choices left open: data structures, persistence, architecture

Data structures. Subjects live in a `HashMap<String, Subject>` keyed by subject code, because lookup is the one thing every operation does and it is $O(1)$. The map is a plain `HashMap`, read by every handler thread without synchronisation, which is safe for a specific reason rather than by luck: it is fully populated before the first handler thread is created and `Thread.start()` establishes a happens-before edge, so every handler sees the completed map, and it is never re-keyed, because no operation adds or removes a subject. All mutable state sits *inside* each `Subject`, which is exactly what the per-subject lock guards. Each subject's students are a `LinkedHashSet<String>`: $O(1)$ membership tests, which is what the `DUPLICATE_ENROLMENT`, `NOT_ENROLLED` and transfer checks need, with insertion order preserved so `QUERY` returns a stable list. A `List` would make every check $O(n)$; a plain `HashSet` would make `QUERY` order arbitrary.

Persistence format. The state file reuses the data file's JSON schema, so one loader and one set of validation rules serve both the first load and every restore. Its path is derived from the data file (`subjects-sample.json` → `subjects-sample-state.json`, same directory) so each data file's state sits beside it instead of in a fixed location two servers would fight over; the original file is never written. Writes go to a `.tmp` sibling that replaces the target with `ATOMIC_MOVE`, so a crash mid-save cannot leave a truncated file. Rewriting the whole file per write is $O(n)$ in subjects rather than $O(1)$ — an append-only log would scale better — but a catalogue is kilobytes, and a full rewrite is far easier to argue correct than a log that must be replayed and compacted.

The subtle part is *what* gets written. Walking the live subjects at save time is wrong in the presence of `TRANSFER`: taking each read lock in turn lets a transfer commit between two of those reads, so the file shows the student in both subjects or in neither — precisely the state no observer may see — while holding every lock at once would serialise the whole server. Each mutation therefore calls `recordState(subject)` *while still holding its write lock*, storing that subject's committed snapshot in `persistedView`; `TRANSFER` records both subjects under both locks, so the saved pair is always mutually consistent. `persist()` encodes and writes that view under a separate `persistMonitor` once the subject locks are released, keeping file I/O out of the per-subject critical sections and ensuring two concurrent saves cannot land out of order. Lock ordering is strictly *subject locks* → *persistMonitor*, so that monitor is a leaf and cannot join a cycle.

Server architecture. One process owns all state, so consistency is a local concurrency problem rather than a distributed-agreement one. The protocol is line-delimited JSON over one long-lived TCP connection per client, strictly one response per request, which keeps the client's `exchange()` a simple synchronous write-then-read. Durability is ordered ahead of acknowledgement: `persist()` runs before the response is written, so a client told `SUCCESS` can rely on the change having reached disk, and a failed save returns `ERROR` saying the change was applied but not saved. Shutdown is ordered too: `quit` closes the listening socket and waits until `inFlightOperations` reaches zero, so running operations still answer their clients while no new one starts — which is not the same as waiting for clients to disconnect, so an idle admin cannot hold the server open.

4.2 Concurrent reads, exclusive per-resource writes, and the last-seat race

The resource is the subject, so the lock is per subject: every `Subject` owns a `ReentrantReadWriteLock` reached through `getLock()`. Keeping the lock *on the object it guards*, rather than in a parallel `Map<String, Lock>`, makes it impossible to acquire the wrong lock for the subject about to be touched and impossible for the two structures to drift apart. `QUERY` takes the read lock, which is shared, so concurrent queries of one subject proceed together; the four write operations take the write lock, which excludes readers and other writers on that subject; and different subjects are different lock objects, so they never contend. The last-seat race is then prevented by making the check and the mutation one indivisible step rather than by adding machinery: `enrol` performs the duplicate check, the capacity check *and* the insertion inside a single write-lock hold.

```
writeLock.lock();                                // abridged: ids = subject.getEnrolledStudentIds()
try {
    pause();                                     // artificial delay, lock held
    if (ids.contains(studentId))                 return DUPLICATE_ENROLMENT;
    if (ids.size() >= subject.getCapacity())     return FULL;
    ids.add(studentId); recordState(subject);    // check and take: one critical section
} finally { writeLock.unlock(); }
```

There is no window between "a seat is free" and "the seat is taken" in which another thread can observe or act on the stale count. The loser of a race necessarily acquires the lock after the winner released it, reads the updated count and gets `FULL`; the lock also supplies visibility — releasing it happens-before any later acquisition — so the loser cannot see a stale cached copy of the set. Measured with a 1000 ms delay, eight clients fired simultaneously at a subject with one free seat gave **1 SUCCESS and 7 FULL** with responses one second apart, while four concurrent `QUERY`s on one subject all returned in **1.02 s** (4 s if serialised) and an `ENROL` concurrent with a `QUERY` on another subject took **1.03 s**.

4.3 Deadlock-free TRANSFER and the choice of lock order

`TRANSFER` holds the write locks of *both* subjects across its entire check-and-move, so no other client can observe the student in both subjects or in neither. Holding two locks is what creates the deadlock risk: two administrators transferring between the same pair in opposite directions — $A \rightarrow B$ and $B \rightarrow A$ — would, under the obvious source-then-destination order, each take their own source lock and then block forever on the other's, a textbook circular wait. The fix is to remove the cycle

by imposing a *total order* on acquisition that does not depend on the request; the locks are taken in ascending `subjectCode` order:

```
boolean fromIsFirst = fromSubjectCode.compareTo(toSubjectCode) < 0;
Subject firstLocked = fromIsFirst ? from : to;
Subject secondLocked = fromIsFirst ? to : from;
```

Both threads now request the *same* lock first: one wins outright, the other waits while holding nothing the winner needs, so no cycle can form. More generally, every thread acquires locks in increasing order along one global ordering, so a thread only ever waits on a lock greater than any it holds, whereas a wait-for cycle would require one to wait on a smaller one. `subjectCode` was chosen as the key because it is the subjects' primary key: unique by validation at load time, immutable for the process's lifetime, totally ordered by `String.compareTo`, and visible in the request itself, which makes the invariant easy to check when reading the code; `System.identityHashCode` is not guaranteed distinct, and creation order would be an ordering invisible at the call site. A self-transfer is rejected with `INVALID_REQUEST` before any lock is taken, so a subject is never locked twice in one operation. Measured with a 1000 ms delay, twenty transfers alternating direction across the same pair and released together all succeeded (**20/20**, no hang) in 20.0 s — exactly the twenty non-overlapping windows the locking predicts.

4.4 Pros and cons of the thread model

The server is **thread-per-connection**: each accepted socket gets its own `java.lang.Thread`, serving that client until it disconnects. That fits what a client actually is here — an interactive console that connects once and then sits idle at a prompt between commands — so the unit of work that persists is the connection, not the request; and since reading a request means blocking in `readLine()` until the admin types, a thread is committed to the connection for its duration under any blocking-I/O model anyway. *Pros*: the handler is straight-line synchronous code that is easy to reason about and get right; a delayed operation on one connection cannot hold up another; no queueing layer adds latency. *Cons*: cost scales with connected clients rather than with useful work, so every idle admin holds a thread stack; the count is unbounded, so a hostile client population could exhaust resources (a cap would be the mitigation); and context-switching grows once threads greatly outnumber cores. At the expected scale that is a reasonable trade, and the first thing to revisit at thousands of connections.

Thread-per-request was rejected because a connection is a stream, not a datagram: something must already be blocked on `readLine()` to know a request has arrived, so the model needs a reader thread per connection *anyway* and adds a second thread per request on top. A *fixed worker pool* bounds thread count, which is right when connections vastly outnumber cores, but with blocking reads a pool of size N is starved by N idle-but-connected clients: the $(N+1)$ th admin cannot be served until someone disconnects. Escaping that needs non-blocking I/O (`java.nio` selectors), a much larger design than this problem warrants.

4.5 The artificial delay: purpose, placement, and its effect on the design

Purpose. The delay argument (`<artificial-delay-ms>`, the server's third argument) stretches every critical section so that concurrency behaviour becomes *observable*. Operations on an in-memory map finish in microseconds, so a broken lock discipline — a race window between check and update, a read needlessly blocking another read, unrelated subjects contending — would almost never be caught by hand, because the window is too small to hit. With a one-second delay held across the lock, "these ran together" versus "these were serialised" becomes a wall-clock measurement from the client rather than an assertion.

Where it is applied. In exactly one place: the private `pause()` helper, called from each of the five operation methods *after* the lock (or, for `TRANSFER`, both locks) has been acquired and *before* any subject state is read or mutated. Every operation has the same shape:

```
acquire lock(s) → pause() → read / mutate → release lock(s) → persist() → send response
```

The sleep is never before acquisition (which would delay without holding anything, making contention invisible) and never after release, and the socket write happens in the connection handler after `process()` returns, so no lock is held across network I/O. One consequence is documented rather than hidden: requests that short-circuit before any lock is taken — `NOT_FOUND`, and `TRANSFER`'s same-subject `INVALID_REQUEST` — are not delayed, since a subject that does not exist has no lock to hold the delay across.

How it shaped the design. Holding a lock for a whole second makes decisions about lock *scope* and *granularity* consequential rather than academic, and drove three of them. First, granularity: under a single global lock a one-second `QUERY` would block an unrelated subject's `ENROL` for that whole second, so per-subject locks and a shared read lock became the difference between a usable and an unusable server. Second, what must *not* be inside the critical section: file I/O is kept outside the subject locks, because a delay-lengthened critical section makes any avoidable work inside it expensive. Third, deadlock exposure: a one-second window with two locks held makes the opposing-transfer interleaving trivially reproducible instead of vanishingly rare, which is what makes the global lock ordering demonstrably necessary and testable. It also turned graceful shutdown into a real design problem — `quit` may have to wait a second for an operation to answer — hence the in-flight operation counter rather than simply closing sockets.